Two-Lane Market Quote Checker

Suggested time: 30 minutes Deliverables: four SystemVerilog source files

Problem statement

Design a combinational circuit that evaluates two independent market quotes in parallel. Each quote contains an unsigned bid price (the price a buyer offers) and an unsigned ask price (the price a seller requests). Determine whether each quote satisfies a configurable maximum spread.

Input format and required behavior

Each quote is a 16-bit word: bits `[15:8]` contain the bid and bits `[7:0]` contain the ask. Both prices are unsigned integers from 0 to 255, expressed in the same arbitrary price unit. A quote passes if and only if

$$\text{bid} \leq \text{ask} \qquad \text{and} \qquad \text{ask} - \text{bid} \leq \text{MAX_SPREAD}.$$

The spread comparison is inclusive. A quote with bid greater than ask must fail regardless of subtraction behavior. The two lanes must operate independently. The circuit must contain no clock, reset, latches, or stored state. Assume all input bits are known 0 or 1 values.

Required module interface

Item	Name	Requirement
Module	`quote_checker`	Combinational, synthesizable RTL.
Parameter	`MAX_SPREAD`	Type `price_t`; default value 5.
Input	`i_quotes`	Unpacked array indexed `[0:1]` of `quote_t`.
Output	`o_pass`	Packed `logic [1:0]`; bit i is 1 exactly when quote i passes.

Source files and responsibilities

- `quote_types_pkg.sv`: Define package `quote_types_pkg`, containing unsigned four-state types `price_t` (8 bits) and `quote_t` (16 bits).
- `quote_utils_pkg.sv`: Define package `quote_utils_pkg`. Import and explicitly re-export both shared types. Define automatic function `quote_passes`, taking bid, ask, and maximum spread as `price_t` arguments and returning a one-bit `logic`. It must depend only on its arguments and use no shared mutable package variables or static local variables.
- `quote_checker.sv`: Implement the required interface. Import the utilities package, extract the prices with part-selects, and evaluate both lanes using the package function and continuous assignments.
- `quote_checker_tb.sv`: Instantiate the DUT with named port connections. Apply stimuli, allow combinational settling time, and automatically check both output bits. Report mismatches with `$fatal`; print a success message and terminate when all checks pass.

Compile in the order listed above. No board implementation or synthesis run is required.

Minimum verification requirements

For `MAX_SPREAD = 5`, check all cases below. Also test one lane passing while the other fails, then swap the lane assignments.

Bid	Ask	Expected	Condition
100	103	Pass	Spread below limit
100	105	Pass	Spread exactly at limit
100	106	Fail	Spread above limit
105	100	Fail	Bid exceeds ask
100	100	Pass	Zero spread
0	0	Pass	Lowest prices
250	255	Pass	Valid quote near upper boundary
0	255	Fail	Maximum possible spread

Evaluation criteria: Functional correctness, unsigned arithmetic, independent lane mapping, package organization, synthesizable coding style, and self-checking verification.